



ТЕХНОЛОГИЧНО УЧИЛИЩЕ "ЕЛЕКТРОННИ СИСТЕМИ"
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

ДИПЛОМНА РАБОТА

по професия код 481020 „Системен програмист“
специалност код 4810201 „Системно програмиране“

Тема: SUPER-GIANT – система за подготовка, трениране
и ускорен инференс на езикови модели

Дипломант:
Антон Иванов Христов

Дипломен ръководител:
Евгени Димов

СОФИЯ

2026

УВОД

През последните години големите езикови модели (Large Language Models, LLM) се превърнаха в основен компонент в съвременните софтуерни системи за търсене, анализ, програмиране и диалогови интерфейси. Паралелно с това нарасна и нуждата от практически използвана отворена имплементация, която да позволява не само обучение и инференс, но и инженерна устойчивост: повтаряеми експерименти, работа на ограничен брой GPU ресурси, преносимост между локална и отдалечена среда и ясно структурирани артефакти.

Настоящата дипломна работа е фокусирана върху разработката на системата SUPER-GIANT, състояща се от две основни части:

- базова LLM платформа GIANT/v2 (данни, модел, обучение, инференс);
- разширение TiDAR, което използва базовата инфраструктура и надгражда обучението и декодирането със забързан diffusion-ориентиран подход, базиран на научния труд “Think in Diffusion, Talk in Autoregression” от NVIDIA Research и надграждането му със собствени модификации.

Реализацията е изцяло на Python, върху JAX стек, с фокус върху:

- скалируем data pipeline с шардове и лесна интеграция с големи хъбове за данни като Hugging-Face или Kaggle;
- стабилно обучение с checkpoint/resume и възстановяване на състоянието на data loader;
- ускорен KV-cache инференс;
- инженерни практики за контейнеризация, синхронизация и работа с GPU в облака.

Целта на дипломната работа е да се проектира и реализира цялостен програмен продукт за обучение и използване на езиков модел, който едновременно:

- осигурява стабилна базова архитектура за autoregressive обучение (GIANT/v2);
- позволява бързо експериментиране с нови training objectives и decoding схеми (TiDAR);
- остава практически приложим в реални инженерни условия (CI/CD, remote execution, cache/sync, reproducibility).

Основни задачи на дипломната работа:

- проучване на съществуващи LLM системи и инструменти;
- описание на архитектурата, алгоритмите и вътрешния модел на данните;
- реализация на ключовите модули в GIANT/v2 и TiDAR;
- валидиране чрез експерименти, измервания на производителност и потребителски сценарии.

ПЪРВА ГЛАВА

ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ СИСТЕМИ, ПОДОБНИ НА GIANT

В тази глава са разгледани решения и тенденции, които са пряко свързани с тематиката на дипломната работа: обучение и ускорен инференс на decoder-only езикови модели.

През последните години езиковите модели преминаха от изследователски прототипи към масови потребителски продукти. Появата на ChatGPT и последвалите системи (GPT семейство, Claude, Gemini, open-source LLaMA и производни) промени очакванията на потребителите: моделът трябва едновременно да е полезен, контекстно точен и достатъчно бърз за интерактивна работа. Именно това поставя фокуса върху архитектури и decode техники, които подобряват скоростта без рязък компромис в качеството.

В исторически план преходът може да се обобщи така: статистически езикови модели -> рекурентни мрежи -> Transformer (2017) -> мащабирани foundation модели. След Transformer централният въпрос вече не е само “може ли моделът да генерира”, а “може ли да генерира достатъчно качествено и достатъчно бързо за реална употреба”.

1.1 Базов контекст: какво е езиков модел

Езиковият модел е софтуерен модел, който предсказва вероятността за следваща дума в текстова последователност. На практика това позволява генериране на нов текст, дописване на изречения и отговори в чат сценарии.

В тази работа се използва decoder-only подход: моделът вижда предишния контекст и предсказва следващия токен. По време на обучение параметрите се настройват итеративно, така че грешката между очакван и предсказан изход да намалява. По време на inference параметрите не се променят, а моделът само генерира.

1.2 Термини (кратки дефиниции)

За целите на дипломната документация по-долу са дадени кратки дефиниции на основните термини. След първото им обяснение те се използват в текста и на английски.

Суперскриптните индекси насочват към материал в раздела “Използвана литература”.

- autoregressive (AR)¹ - режим, при който моделът генерира следващ токен на база на предходните токени;
- inference² - процесът на използване на вече обучен модел за генериране, без трениране;
- KV cache² - кеш за Key/Value матриците в attention слоя, който ускорява decode стъпките;

- `prefill`² - начален forward pass върху prompt-a, който запълва KV cache преди decode;
- `decode`² - итеративна фаза на генериране на нови токени след prefill;
- `sharding`³ - разделяне на голям dataset на по-малки shard файлове за по-ефективно четене;
- `checkpoint`⁴ - запис на състоянието на модела и оптимизатора в конкретна training стъпка;
- `resume`⁴ - продължаване на training от съществуващ checkpoint вместо start от нула;
- `stage`⁵ - отделен обучителен етап с конкретни настройки (данни, дължина, loss коефициенти);
- `curriculum`⁵ - планирана последователност от stages с последователна сложност
- `loss function`¹ - измерва грешката на модела и определя посоката на оптимизация;
- `optimizer`⁶ - алгоритъм за обновяване на параметрите по посоката на градиент (AdamW¹⁴);
- `batch`⁹ - група от примери, обработвани заедно в една training стъпка;
- `epoch`⁹ - едно пълно преминаване през тренировъчния набор данни;
- `throughput`⁷ - скорост на обработка (например tokens/s);
- `latency`⁷ - време за единична операция/отговор (например време за decode итерация);
- `benchmark`² - стандартизирано измерване на производителност при фиксирани условия;
- `artifact`⁴ - генериран резултат от pipeline/training/inference (лог, checkpoint, графика, отчет);
- LLaMA⁸ - семейство от големи езикови модели (Meta), използвани като архитектурен ориентир;
- LayerNorm¹³ - нормализация на активации чрез средна стойност и стандартно отклонение;
- RMSNorm¹⁰ - нормализация на активации по RMS стойност;
- RoPE¹¹ - rotary позиционно кодиране за attention;
- SwiGLU¹² - gated feed-forward активация, често използвана в модерни LLM;
- `serving`² - продукционен режим за подаване на заявки към вече обучен модел.

1.3 Сходни технологии и инженерни практики

1.3.1 LLaMA семейство (Meta)

LLaMA моделите⁸ се утвърдиха като де факто стандарт за отворени foundation модели. Основните им предимства са добра scaling стратегия, стабилни архитектурни избори (RMSNorm, RoPE, SwiGLU) и голяма екосистема от производни реализации. За проекта GIANT тези модели са референтна точка за архитектурни и обучителни практики.

1.3.2 SmolLM / компактни open модели

SmolLM профилите са важни като practical baseline за експерименти с ограничен GPU бюджет. В TiDAR частта на проекта се използват точно такива размери (135M и 360M), което позволява многократни сравнения между loss конфигурации и decode варианти в разумно време.

1.3.3 nanoGPT / minGPT

Тези проекти показват минимална, ясно четима реализация на autoregressive Transformer. Те са полезни за концептуално разбиране, но не покриват production-like нуждите от sharding, resume, инфраструктура за отдалечен GPU и систематични benchmark отчети.

1.3.4 vLLM и TensorRT-LLM

vLLM² и TensorRT-LLM са системи за ефективно изпълнение на вече обучени езикови модели. Те намаляват закъснението и повишават пропускателната способност чрез оптимизирано управление на KV кеша и GPU изпълнението. В проекта тези практики се използват като инженерен ориентир при оценката на скоростта и при избора на decode политики.

1.3.5 Защо скоростта е критична за LLM приложения

При интерактивни системи за търсене, чат и асистирано писане скоростта не е “удобство”, а основна част от качеството на продукта. По-високата latency директно намалява вероятността потребителят да продължи взаимодействието.

Класически пример е web search: експериментите на Google (Brutlag, 2009) показват, че увеличение на latency от 100 ms към 400 ms води до измерим спад в броя търсения на потребител, а ефектът се натрупва с времето. Това е пряко свидетелство, че дори “малки” закъснения имат реална продуктова цена.

Отделен клас критични приложения са системите за real-time decision making с AI (оперативни асистенти, monitoring и бърз анализ на събития). В тях латентността влияе пряко върху качеството на решението във времето: при закъснял отговор прозорецът за реакция може вече да е затворен. В такива сценарии ускорение от порядъка 2x-5x е стратегически значимо, защото позволява повече reasoning итерации в същия времеви бюджет.

Именно затова в проекта скоростта се разглежда като първокласна инженерна цел наред с качеството на изхода.

Speed Matters for Google Web Search

Jake Brutlag
Google, Inc.
June 22, 2009

Abstract – Experiments demonstrate that increasing web search latency 100 to 400 ms reduces the daily number of searches per user by 0.2% to 0.6%. Furthermore, users do fewer searches the longer they are exposed. For longer delays, the loss of searches persists for a time even after latency returns to previous levels.

Figure 2: Impact of Post-header Delays Over Time

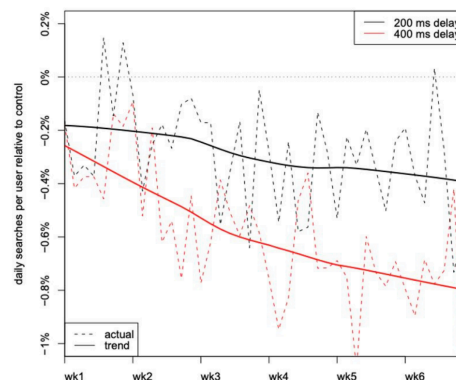


Figure 1: Влияние на latency върху потребителското поведение

при търсене

ВТОРА ГЛАВА

ФУНКЦИОНАЛНИ ИЗИСКВАНИЯ И АРГУМЕНТАЦИЯ НА ИЗБОРА НА СРЕДСТВА ЗА РАЗРАБОТКА

Тази глава описва какво трябва да прави системата, защо е избран точно този технологичен стек и как е организиран вътрешният модел на данните и модулите. Целта е читателят да може да проследи логиката от входните данни до крайния training и inference процес, дори без предишен опит с LLM инфраструктура.

2.1 Функционални изисквания

В контекста на тази работа **обучение** (training) означава итеративно нагласяване на параметрите на модела така, че разликата между очакван и предсказан изход да намалява с всяка следваща стъпка. Това става чрез изчисляване на грешка (loss), обратно разпространение на градиентите и обновяване на параметрите от оптимизатор.

2.1.1 Входни източници и ingest pipeline

Системата трябва да поддържа ingest (контролирано зареждане и уеднаквяване) на текстови корпуси от няколко типа източници:

- публични HuggingFace datasets;
- локални JSON/JSONL директории;
- chat-структурирани данни с role/content полета.

Ingest pipeline е последователност от стъпки, която превръща суровия текст във формат, удобен за обучение: нормализация на текст, токенизация, филтриране и запис в стандартизирани shard файлове.

2.1.2 Подготовка на корпус, токенизация и stage-и

Токенизация означава разбиване на текста на по-малки единици (токени), които моделът реално обработва като числа. Вместо да “чете” директно думи, моделът работи с последователности от token ID стойности.

Системата използва **stages** (обучителни етапи). Всеки stage може да има собствена дължина на последователността (seq_len), дял от данните (fraction) и брой епохи (epochs).

Епоха (epoch) е едно пълно преминаване през избраната част от dataset-a за дадения stage.

2.1.3 Конфигурационно управление на експерименти

Експериментът се описва чрез YAML конфигурации. Това е важно за възпроизводимостта: при същите конфигурации и данни резултатът трябва да бъде максимално близък.

Основните нива са:

- глобални настройки в `Global_Config.yml` (пътища, базова среда);
- локални `model/training` настройки в `Config.yml`;
- stage-level настройки за dataset дялове и loss коефициенти;
- CLI overrides за бързи промени без пренаписване на конфигурационните файлове.

2.1.4 Мониторинг и метрики по време на изпълнение

При дълги run-ове е задължително системата да показва междинно състояние, а не само краен резултат.

Затова се изисква:

- логване на loss и помощни метрики по стъпки;
- acceptance метрики (дял приети draft токени) при TiDAR decode;
- запис на checkpoint metadata (стъпка, конфигурация, време);
- автоматично генериране на benchmark отчети.

Метрика е измерима стойност за качество или производителност. Типични примери са loss, accuracy, tokens/s (throughput) и latency.

2.1.5 Управление на хиперпараметри и decode политика

Хиперпараметри са настройки, които не се научават автоматично от модела, а се задават от разработчика: размер на embedding, брой слоеве, размер на batch, learning rate и др.

В тази система управлението обхваща:

- архитектурни параметри (`embedding_size`, `num_heads`, `num_kv_heads`, `num_layers`);
- числен тип на изчисленията (`compute_dtype`, `param_dtype`);
- контекстна дължина и политика за KV cache;
- TiDAR decode параметри (`draft_length`, attention bias, sampling настройки).

2.1.6 Артефакти, checkpoint-и и отчетност

Артефакт е всеки запазен резултат от изпълнението: файл с параметри, лог, графика, отчет.

Системата трябва да съхранява и поддържа:

- dataset shard-ове + manifest + статистики;
- checkpoint-и на параметри + optimizer/dataloader state;
- inference и измервания на производителност (скорост и закъснение);

- текстови и визуални отчети (.md, .typ, .pdf).

Checkpoint е моментен запис на състоянието на обучението (параметри, optimizer state и metadata) в конкретна стъпка. **Resume** е възстановяване и продължаване на обучението от този запис, вместо нов старт от нула.

2.2 Аргументация за избор на езика и софтуерните средства

2.2.1 Избор на езика за програмиране

Изборът на Python е обусловен от практичността му за ML инженеринг:

- много богата екосистема за data/ML;
- бърз цикъл на прототипиране;
- ясна интеграция със shell автоматизация, Docker и cloud workflow.

2.2.2 Избор на платформа за разработка

Избрана е GPU-ориентирана среда, тъй като обучението и инференсът на LLM са изчислително интензивни и се ускоряват значително чрез паралелната обработка на графичните процесори. Съвременните модели видеокарти разполагат със специализирани изчислителни блокове, които ускоряват матричните умножения - една от основните операции в машинното обучение. Понеже локалната машина не е достатъчна за всички сценарии, работният модел е хибриден:

- локална разработка и подготовка на данни;
- remote GPU изпълнение за тежки run-ове;
- синхронизация чрез Docker + S3 tooling.

Така се постига баланс между цена, скорост и контрол на средата.

2.2.3 Избор на изчислителен фреймуърк

JAX (JIT AutoGrad XLA) е избран заради **JIT** (Just In Time) компилацията и оптимизациите от **XLA** (aXelerated Linear Algebra). На практика Python функцията се трасира и компилира до оптимизиран XLA граф, специализиран за конкретни форми и типове на входа.

Съществена характеристика на JAX е, че следва **чисто функционална парадигма** (pure functional paradigm): изчисленията се описват като функции без скрити странични ефекти, а състоянието се подава явно чрез входове и изходи. Точно тази форма улеснява JIT/XLA трансформациите (компиляция, векторизация, оптимизация на графа) и насочва разработката към optimization-first подход, при който архитектурните и числените решения се проектират с мисъл за ефективно компилируемо изпълнение.

Flax е надстройка над JAX за модулно описание на невронни архитектури и управление на параметри/състояние.

Комбинацията JAX + Flax е ключова за проекта, защото позволява:

- висока производителност;
- ясен модел на train/inference state;
- интеграция с custom KV cache логика и TiDAR маски.

2.2.4 Избор на optimizer/checkpoint инструменти - Orbach, Optax и AdamW

Използваме Orbach за checkpointing. Записват се параметрите на модела, състоянието на оптимизатора и текущата training стъпка, така че обучението да може да продължи коректно след прекъсване. Optax ни предоставя оптимизатор и възможност за прецизен график на скоростта на обучение (learning rate schedule).

Всички training конфигурации използват AdamW¹⁴. Това е модерен gradient optimizer, който ползва momentum, за по-бързо трениране, adaptive learning rate и weight decay.

Momentum, което е експоненциално изгладена оценка на градиента, която стабилизира посоката на обновяване. Adaptive learning rate означава, че всеки параметър получава собствена ефективна стъпка според историята на градиентите. Weight decay - отделен регуляризационен член, който ограничава нарастването на нормата на параметрите и подпомага обобщаването.

2.2.5 Избор на средства за deployment и синхронизация

Използват се Docker, Tailscale и S3 съвместими инструменти.

- Docker осигурява еднаква изпълнима среда (еднакви зависимости) между машини;
- Tailscale осигурява защитена частна мрежа (VPN), базирана на WireGuard протокола, за remote достъп;
- S3 workflow позволява надеждна синхронизация на данни и артефакти между локална и remote среда.

2.3 Структура на данните и вътрешен модел на системата

2.3.1 Формат на данните и логика на съхранение

Системата използва файлово-ориентиран модел, а не релационна база данни.

Основният формат за обучителни данни са Arrow shard файлове, придружени от manifest и статистики. Този избор е направен заради бързо последователно четене от data loader-a и добра съвместимост с големи обеми текст.

JSON/TXT формати се използват като входни или междинни формати в ingest етапа, но не и като финален training storage backend.

- управляващи модули (pipeline, training, checkpointing, inference);
- математически модел (Transformer + TiDAR разширения).

2.4.1 Интерфейсен слой (CLI)

Интерфейсният слой е CLI-first: потребителят работи чрез entry point скриптове и YAML конфигурации, което осигурява директен контрол на параметрите, автоматизация чрез скриптове и възпроизводимост на всеки run. Същият интерфейс се използва еднакво при локална работа и при отдалечено изпълнение през SSH/tmux върху GPU среда.

В текущата реализация не се използва отделно графично табло от тип TensorBoard. Вместо това системата води структурирано логване на метриките по време на обучението (loss, ppl, acceptance показатели и др.), като тези логове се преобразуват директно във фигури и графики чрез Typst. Примерите за такава визуализация са показани в следващите раздели на документа. Критичните метрики, които показват дали даден training run трябва да бъде прекратен или коригиран, се извеждат постоянно на терминалния екран по време на изпълнение в tmux. Това осигурява непрекъснат оперативен контрол без необходимост от отделен графичен интерфейс.

2.4.2 Управляващи модули

Управляващите модули са “гръбнакът” на системата:

- data pipeline модул за build на корпус и shard-ове;
- training оркестратор за stages, optimizer стъпки и логване;
- checkpoint мениджър за save/restore/resume;
- inference runtime за prefill/decode и performance отчети.

TiDAR надгражда тази основа със специализирани маски и допълнителни loss механизми, без да нарушава базовия workflow.

2.4.3 Основни entry points

Ключовите скриптове са:

- GIANT/v2/data_pipeline/build_corpus.py
- GIANT/v2/model/Run_training.py
- GIANT/v2/model/Generate_faster.py
- GIANT/v2/model/Chat.py
- TiDAR/model/Run_training.py
- TiDAR/model/inference.py

Така системата запазва ясна граница между базова платформа (GIANT/v2) и изследователски разширения (TiDAR).

2.5 Основен математически модел на трансформер архитектурата

Този раздел представя концептуално и формално принципа на работа на Transformer архитектурата, върху която е изграден GIGANT. Изложението преминава от структурно описание на компонентите към математическа формализация на основните операции.

2.5.1 Теоретична отправна точка: Attention Is All You Need

По-ранните езикови модели обработват последователно с рекурсия, токен по токен. Тази зависимост ограничава паралелизма при обучение и увеличава времето за обработка на дълги входове.

Архитектурата Transformer¹ е въведена в статията **Attention Is All You Need** през 2017 г.¹.

Transformer адресира това ограничение чрез механизма **attention**, при който за всяка позиция се изчислява релевантност спрямо останалите позиции в контекста и се формира претеглена контекстна репрезентация.

Първите широко използвани Transformer системи са encoder-decoder модели, насочени основно към машинен превод. През 2018 г. BERT утвърждава encoder подход с двупосочен контекст при обучение.

Също през 2018 г. започва линията decoder-only модели (GPT), при които се използва причинно-следствена (causal) маска: токен на дадена позиция може да “вижда” само токени от предходни позиции, не може да “вижда в бъдещето”. Тази autoregressive decoder-only постановка се превръща в архитектурна основа на съвременните LLM системи.

Операционната логика на attention може да се обобщи така:

- моделът сравнява текущата позиция с всички допустими позиции според маската;
- оценява колко е релевантна всяка от тях;
- комбинира информацията според тези оценки;
- предава получената смес към следващите слоеве.

Тази конструкция позволява паралелна обработка на цели блокове от токени и е фундаментална за съвременните LLM системи.

$$S(z)_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Така weights се интерпретират като вероятностно разпределение (сборът им е 1), което прави смесването стабилно и интерпретируемо.

Кратък числен пример. Нека за една позиция logits са:

$$z = (2, 1, 0)$$

Тогава:

- $e^z = (e^2, e^1, e^0) \approx (7.39, 2.72, 1.00)$;
- $Z = \sum_j e^{z_j} = 7.39 + 2.72 + 1.00 = 11.11$;
- $S(z) = \frac{1}{Z}(7.39, 2.72, 1.00) \approx (0.665, 0.245, 0.090)$.

Това означава, че първата позиция получава най-голяма тежест, но не “занулява” останалите напълно. Именно така моделът комбинира няколко релевантни места от контекста, вместо да избира само едно.

Делението на $\sqrt{d_k}$ преди softmax е числено важно: без него при големи размерности на векторите dot-product стойностите стават твърде големи, softmax се “насища” и градиентите стават нестабилни.

Класическата форма от оригиналната публикация е:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

В реалните decoder-only модели почти винаги има маска, затова в инженерната форма се използва и M .

В causal режим маската обикновено се задава така: $M_{i,j} = 0$ при $j \leq i$ и $M_{i,j} = -\infty$ при $j > i$.

Тоест позиция i няма право да използва информация от бъдещи позиции $j > i$.

В приложената диаграма (горе вдясно) това е показано с mask анотация от вида $i < j$: червените клетки са забранени (маскирани), а белите са разрешени. Това визуално е същата причинно-следствена идея: “бъдещето” е скрито за текущата позиция.

2.5.2 Multi-head attention: защо са нужни “много глави”

Ако attention е само един, моделът гледа контекста от една “перспектива”. Multi-head attention въвежда няколко паралелни attention подпространства, които могат да уловят различни типове зависимости едновременно:

- синтактични (напр. кой е подлог/сказуемо);
- семантични (напр. кой обект се отнася към кого);

2.5.4 Нормализация, активации и позиционна информация в GIANT

GIANT използва съвременен decoder-only профил:

- RMSNorm¹⁰ вместо класически LayerNorm¹³
- SwiGLU¹² в feed-forward частта;
- RoPE¹¹ (rotary positional embeddings) за позиционна информация.

RMSNorm¹⁰:

$$R(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} * g$$

RMSNorm държи мащаба на активациите стабилен между слоевете, което подпомага обучението при по-дълбоки модели.

SwiGLU¹² гейтинг:

$$G(x) = s(xW_u) * (xW_v), \quad s(t) = t\sigma(t)$$

SwiGLU работи с два паралелни линейни пътя. Първият път (xW_u) минава през SiLU нелинейност и играе ролята на gate, а вторият (xW_v) носи съдържателния сигнал. Точковото умножение между двата пътя потиска част от каналите и усилва други според контекста.

Това е съществено по-изразително от еднолинейна ReLU MLP схема, защото дава контекстно зависим контрол върху потока на информацията във всеки токенов вектор.

Тук **gate** означава вектор от коефициенти за “пропускане” на информация. След SiLU стойностите в този вектор могат плавно да намалят или увеличат отделни канали. Пътят v е “съдържание” (content branch) - той носи самите признаци, които gate векторът мащабира.

В имплементацията на GIANT (Transformer_block.py) това е реализирано така:

- fc1: проекция от `d_model` към `2 * d_ff`;
- split на две равни части u и v (по `d_ff` неврона всяка);
- `gate = SiLU(u)`;
- `h_ffn = gate * v` (елементно умножение);
- fc2: проекция обратно от `d_ff` към `d_model`.

Числени примери по реални конфигурации:

- TinyStories/TiDAR базов профил (`d_model=384`, `d_ff=1024`): fc1 дава 2048 канала, split до $u = 1024$ и $v = 1024$, после fc2 връща към 384;
- GIANT 135M профил (`d_model=576`, `d_ff=1536`): fc1 = 3072, split 1536 + 1536, после fc2 = 576;

- целеви токен: `Paris` с integer label `y = id("Paris")`;
- ако моделът дава $p(y) = 0.62$, тогава $l = -\ln(0.62) \approx 0.478$;
- ако моделът дава $p(y) = 0.05$, тогава $l = -\ln(0.05) \approx 2.996$.

Разликата показва защо cross-entropy е подходяща за езиково моделиране: системата получава много по-голям сигнал за корекция, когато правилният токен е оценен с ниска вероятност.

Концептуална визуализация на AR next-token предсказване

Контекст: The capital of France is Целеви етикет: `y = id("Paris")`

Следващ токен (вероятности след softmax):

Paris	: #####	0.62	<- целев клас
London	: #####	0.21	
Lyon	: ##	0.04	
other	: #####	0.13	

Listing 1: Пример за разпределение върху речника при AR обучение (по идея на визуализациите за next-token prediction)

Примерен изход от GIANT v2 checkpoint, трениран върху корпус със силно Wikipedia присъствие (заедно с други източници), е:

User: What is the capital of France?

Assistant: The capital of France is sometimes called Paris.<EOS>

- размер на модела: приблизително 101 милиона параметъра;
- обем на обучението: приблизително 2 милиарда токена.

За мащабно сравнение:

- Llama 3 70B - trained on 15T tokens;
- DeepSeek R1 671B - trained on 14.8T tokens.

TiDAR стъпва директно върху тази основа. Тоест първо се учи стабилен autoregressive модел, а след това се добавят TiDAR разширенията (masking логика и допълнителни loss термини) за по-ефективен decode.

2.6 Спекулативно декодиране (въведение преди TiDAR)

Преди представянето на TiDAR е важно да се въведе идеята за speculative decoding¹⁶, защото именно тя е базата, върху която стъпва по-късното Anchor-TiDAR разширение.

Класическото autoregressive декодиране генерира по един токен на стъпка. Това е коректно, но често не използва оптимално паралелните възможности на GPU при inference.

ТРЕТА ГЛАВА

РЕАЛИЗАЦИЯ НА ПРИЛОЖЕНИЕТО

3.1 Реализация на потребителски интерфейс

3.1.1 Основни входни точки (CLI)

Системата е CLI-first и основният интерфейс е набор от entry points:

- `GIANT/v2/data_pipeline/build_corpus.py` - build и шардване на корпус;
- `GIANT/v2/model/Run_training.py` - обучение по stage конфигурации;
- `GIANT/v2/model/Evaluate.py` - офлайн оценка на checkpoint;
- `GIANT/v2/model/Generate_faster.py` - ускорен decode и benchmark;
- `GIANT/v2/model/Chat.py` - интерактивен чат inference;
- `TiDAR/model/Run_training.py` - TiDAR обучение и логване;
- `TiDAR/model/inference.py` - TiDAR inference и сравнения.

Тези скриптове формират основния интерфейс на системата и покриват пълния цикъл: data prep, обучение, оценка и инференс.

3.1.2 Наблюдение и визуализация на резултати

Визуализацията се реализира чрез логове и експериментални отчети:

- training логове (loss, ppl, accept, greedy_accept);
- checkpoint metadata (train_loss, val_loss, val_ppl);
- Typst отчети с throughput и policy сравнения.

Този подход е избран целенасочено за terminal/vim workflow и за лесно наблюдение през SSH.

3.1.3 Управление на stage прогреса

Управлението на прогреса е реализирано чрез stage curriculum. Изпълнението преминава през последователни stages с различен dataset и контекстна дължина, като за всеки етап се съхранява възстановимо състояние.

3.1.4 Управление на параметри

Управлението на параметрите е изнесено в YAML конфигурации и CLI flags. Това позволява силно контролирани експерименти с минимални кодови промени.

3.1.5 Добавяне на входни корпуси

Pipeline-ът поддържа HuggingFace и локални JSON/JSONL входове, chat formatting и loss-mask логика.

3.1.6 Експорт на артефакти

Експорт на артефакти към файловата система/S3:

- Arrow shards;
- manifests/statistics;
- checkpoints;
- benchmark logs;
- Typst/PDF отчети.

3.1.7 Примерен training run в терминал

Следният пример показва типично стартиране на обучение през CLI интерфейса:

```
python GIANT/v2/model/Run_training.py \  
  --config GIANT/v2/model/Config.yml \  
  --global_config GIANT/v2/Global_Config.yml \  
  --checkpoint_dir giant-data/GIANT/v2/checkpoints \  
  --checkpoint_every 500 \  
  --scan_chunk 8
```

Пример за възстановяване на последната налична checkpoint точка:

```
python GIANT/v2/model/Run_training.py \  
  --config GIANT/v2/model/Config.yml \  
  --global_config GIANT/v2/Global_Config.yml \  
  --checkpoint_dir giant-data/GIANT/v2/checkpoints \  
  --resume latest
```

Примерен терминален изход при старт от нула (съкратен):

```
[loader] loading shards from /proj/giant-data/GIANT/v2/processed/stage_base  
...  
[loader] stage=base rows=12800000/12800000 ctx=1024 steps_per_epoch=6250  
  
→ Stage base: seq_len=1024 epochs=1 steps=6250 (resume at 0)  
step    100/10449   | stage base           loss 4.8921 ppl 133.21 (18.4s)  
step    200/10449   | stage base           loss 4.6017 ppl 99.66 (17.9s)  
...  
step    500/10449   | stage base           loss 4.2143 ppl 67.64 (17.2s)  
[metadata] Wrote train_loss=4.214300 to checkpoint .../params/step_0000500.npz  
📁 checkpoint → .../params/step_0000500.npz
```


Примерен терминален изход при `--resume latest` (примерно от стъпка 5432):

```
↳ Resumed from mini checkpoint at step 5432
▶ Restored dataloader state at stage 0 step 5432

→ Stage base: seq_len=1024 epochs=1 steps=6250 (resume at 5432)
step    5500/10449   | stage base                loss 3.9981 ppl 54.50 (16.9s)
...
→ Stage wiki: seq_len=1024 epochs=1 steps=4199 (resume at 0)
step    6400/10449   | stage wiki                 loss 3.7129 ppl 40.97 (16.8s)
...
✓ Training complete. Final checkpoint: .../params/step_0010449.npz
```

Примерен multi-stage TiDAR training run с различни loss конфигурации по етапи:

```
python TiDAR/model/Run_training.py \
  --config TiDAR/model/training_configs/Greedy_exp_135m.yml \
  --global_config TiDAR/Global_Config.yml \
  --checkpoint_dir giant-data/TiDAR/checkpoints \
  --checkpoint_every 400 \
  --resume latest
```

```
[loader] loading shards from /proj/giant-data/TiDAR/processed/stage_base
...
[loader] stage=base rows=6200000/6200000 ctx=1024 steps_per_epoch=3027
[loader] loading shards from /proj/giant-data/TiDAR/processed/stage_chat
...
[loader] stage=chat rows=1800000/1800000 ctx=1024 steps_per_epoch=878

→ Stage base: seq_len=1024 epochs=1 steps=3027 (resume at 0)
step      1/3905    | stage base                loss 5.1023 ar 4.8122 diff
5.2761 accept 0.117 greedy_acc 0.081 (19.2s)
step     200/3905    | stage base                loss 4.4311 ar 4.1926 diff
4.5214 accept 0.223 greedy_acc 0.147 hard 3.8872 (17.4s)
...

→ Stage align_kl: seq_len=1024 epochs=1 steps=1000 (resume at 0)
step     3200/3905   | stage align_kl            loss 3.9027 ar 3.7710 diff
```

3.2.5.5 Метод BatchedPrefill

`test_batched_inference.py` валидира `batched prefill` с отмествания и проверява съвпадение с `per-sample baseline`.

3.2.5.6 Метод StopOnEos

Генерацията може да прекъсва на EOS токен за контрол на отговора и избягване на нежелано продължение.

3.2.5.7 Метод ThroughputReport

Отчетите измерват `prefill/decode` време, `tokens/s` и `policy` сравнения по контекст/модел/драфт параметри.

3.2.6 RemoteOpsManager

CICD/tools/ и Docker setup покриват `deployment` и `remote execution`. Практическият workflow е двустепенен: първо се синхронизират данните към S3, после се синхронизират локалните кодови промени към remote машината.

Примерен remote workflow

1) sync данни/артефакти към S3

```
python CICD/tools/s3.py sync giant-data s3://bucket/giant-data
```

2) sync локални промени по репото към remote GPU машината

```
python CICD/tools/sync-gpu.py
```

3.2.6.1 Основен принцип на работа

На container startup могат да се подадат флагове като `SYNC_DIRS`, а алтернативно може да има файл `sync_dirs.txt` в root-a на S3 bucket-a. Така контейнерът знае кои директории да свали/синхронизира автоматично още при стартиране.

След това `sync-gpu.py` изпраща локалните (вкл. непущнати/несommit-нати) промени по кода към remote репо копието. Remote машината стартира от последния commit-нат код, а `sync-gpu.py` донася текущите локални редакции от работната сесия.

3.2.6.2 Кодова синхронизация преди run

Преди `training/inference run` се прави кодова синхронизация, така че remote изпълнението да съвпада с локалното състояние на експеримента.

Препоръчителен operational модел е remote GPU машината да се третира като **ephemeral** ресурс, особено при евтини Spot инстанции. Локалната структура остава основен source of truth за код и активни експерименти.

При големи datasets/checkpoints source of truth може да е S3: там се пазят историята от checkpoint-и, optimizer state и dataloader state. На remote машината се изтеглят само нужните

```
rows_per_shard: int | None = None
sources: list[StageSourceCfg] = field(default_factory=list)
```

3.3.2 Преобразуване от документ към sequence прозорци

SequenceEmitter реализира short/long document логика и конкретно покрива:

- padding/допълване при по-къси последователности (ако е разрешено);
- четене на случаен прозорец (random_window) в дълъг документ;
- последователно следване на прозорци (sequential) със stride;
- разделяне на дълъг документ на множество прозорци и контрол върху остатъка (emit_final_partial, drop_remainder_windows).

Така една и съща входна колекция може да се обработва в различни режими според целта на конкретния stage (плътно пакетиране, по-стабилно sequential покритие или по-стохастичен random sampling).

3.3.3 Директно четене и минимизиране на IO overhead

Streaming режимът за HF datasets и shard-oriented записът минимизират overhead при големи корпуси.

3.3.4 Шардване

Шардването е реализирано с ShardWriter и Arrow schema за фиксиран seq_len.

3.3.4.1 Формат на шардовете

input_ids, length, loss_mask се записват в .arrow файлове с manifest по stage.

3.3.4.2 Manifest и статистики

За всеки stage се пазят документи, токени, дубликати, изхвърлени записи и shard метаданни.

3.3.4.3 Контрол на паметта

Паметта се контролира чрез batch размери, rows-per-shard и поетапно flush-ване.

3.3.5 Loader batch изграждане

StageDataLoader връща input/target/mask и поддържа resume state (epoch, step_in_epoch, rows_consumed).

3.3.5.1 Prefetch като оптимизация

_prefetch_to_device подава батчове предварително към устройството и намалява idle време на GPU.

3.3.6 Допълнителни preprocessing опции

Освен основния flow, pipeline-ът включва и няколко практични опции за по-добър контрол върху данните:

- assistant-aware chat маски, така че loss да се натрупва само върху целевите роли;

$$L = \alpha L_A + \beta L_D + \rho D_f + \chi D_r + \delta L_H + \delta_m L_P + \eta L_S + \gamma L_K$$

За сравнимост между различни конфигурации с различна обща тежест на коефициентите, в анализа може да се използва и нормализирана форма, при която относителните дялове се запазват:

$$w_i = \frac{\lambda_i}{\sum_j \lambda_j + \varepsilon}, \quad L_{\text{norm}} = \sum_i w_i T_i$$

Тук T_i обозначава съответния loss термин, а ε е малка числена константа за стабилност.

Където $|(P_A)$ означава AR разпределение със stop-gradient (без обратен градиент към AR клона).

- AR термин:

$$L_A = -\frac{1}{S-1} \sum_{t=0}^{S-2} \log P_{A,t}(x_{t+1})$$

Поддържа стандартната next-token езикова способност в clean половината.

- Diff термин:

$$L_D = -\frac{1}{S-1} \sum_{t=0}^{S-2} \log Q_{D,t}(x_{t+1})$$

Учи diffusion половината да реконструира същата бъдеща цел от mask-нат вход.

- Forward KL термин:

$$D_f = \sum_v |(P_A(v)) \log \frac{|(P_A(v))|}{Q_D(v)}$$

Наказва Diff, когато изпуска вероятностна маса, която AR счита за важна.

- Reverse KL термин:

$$D_r = \sum_v Q_D(v) \log \frac{Q_D(v)}{|(P_A(v))|}$$

Наказва Diff, когато разпределя излишна маса извън AR разпределението.

Съвместното използване на D_f и D_r дава по-стабилно подравняване от използването само на един KL термин. D_f пази покритието на важните AR кандидати, а D_r ограничава вероятностната маса в слабо релевантни опашки. На практика това позволява по-фин баланс между recall на валидни токени и селективност на Diff разпределението чрез коефициентите ρ и χ .

- Hard agreement термин:

$$L_H = -\log Q_D(v^*)$$

Притиска greedy избора на Diff да съвпада с greedy избора на AR; v^* е токена с най-висока вероятност според AR.

- Masked hard agreement термин (prefix mask):

$$L_P = \frac{1}{N_P} \sum_{i \in P} -\log Q_{D,i}(v_{A,i}^*)$$

Това е новият “маскиран” hard loss: взема същия hard сигнал, но само върху позициите до първото несъвпадение между AR и Diff (включително). В кода този термин се управлява с отделен коефициент `delta_masked` и служи да фокусира натиска върху префикса, който е най-важен за асерт логиката.

- Soft distillation термин:

$$L_S = T^2 D(p_A^T \| p_D^T)$$

Пренася “меката” форма на AR разпределението към Diff, без да се фиксира само един токен; p^T са температурно-скалирани вероятности.

- Top-K set distillation термин:

$$L_K = -\log \sum_{v \in V_K} Q_D(v)$$

Концентрира вероятностната маса на Diff в топ-K набора на AR и подобрява шанса за асерт при decode; V_K е множеството от топ-K AR кандидати.

Важно техническо поведение в кода: при коефициент 0 съответният термин се пропуска напълно от изчислението (`alpha`, `beta`, `rho`, `chi`, `delta`, `delta_masked`, `eta`, `gamma`), което позволява експериментите да се правят с минимален излишен compute и с ясна изолация на ефекта от всеки компонент.

Механизъм на JAX компилацията: в `TiDAR/model/Run_training.py` функцията `_run_chunk` е JIT-компилирана чрез `@partial(jax.jit, static_argnames=...)`, като loss коефициентите са подадени като static аргументи. При нова комбинация от стойности се компилира нов изпълним вариант, а при повторение на същата комбинация се използва кешираният вариант. Тъй като branch-овете в `TiDAR/model/Training_step.py` са условни по коефициент, изключените термини се `prune`-ват от съответния `jaxpr`.

3.8 Резултати от greedy run-ове (135M vs 360M)

Тази секция обобщава резултати от реално тренирани TiDAR варианти с `draft_len=8`: 135M и 360M. Данните са от продължителен run (около 5 часа), при който по-големият модел е стартиран с по-голям batch, а логовете са подравнени по optimizer стъпки.

```
seq_len: 512
epochs: 3
fraction: 0.75
```

Примерни изречения/промпт фрагменти от корпуса:

- “There was a small village.”
- “Once upon a time, a little girl...”
- “In the forest, a tiny fox...”
- “One day, the teacher said...”
- “The robot wanted to learn...”

3.10.1 Експериментален протокол и избор на checkpoint

Основният TiDAR baseline run ($\alpha=1$, $\beta=1$) достига 121566 optimizer стъпки. За сравнителната част е избран checkpoint около 90k, от който се пускат над 10 различни loss конфигурации до приблизително 121k, за да се измери ефектът им върху Diffusion/Greedy метриките при близки стартови условия.

Изборът на 90k е целенасочен: в този етап baseline режимът започва видимо да забавя нормалното си учене при $\alpha=\beta=1$, което го прави подходяща точка за branch сравнения. Част от експериментите са стартирани и от нулева стъпка за допълнителна проверка на ранната динамика.

Branch-based дизайнът е силен за сравнение, защото:

- елиминира ефекта от различна ранна инициализация;
- сравнява режимите върху близка checkpoint основа;
- позволява директни delta криви спрямо baseline по една и съща step ос.

Във файла са използвани два прозореца на анализ:

- пълен прозорец (за стабилния run) 0-121k за глобална динамика;
- zoom прозорец 90k-121k за head-to-head сравнение между branch вариантите.

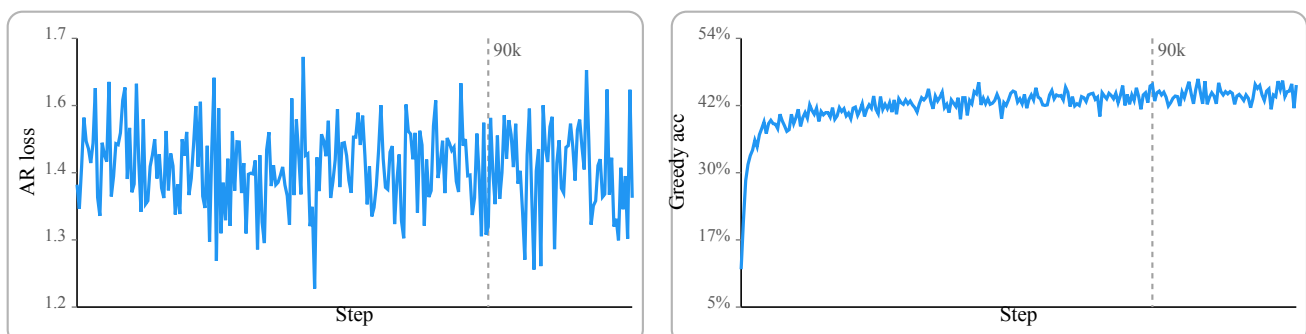


Figure 28: Базова траектория и branch cut при 90k стъпка

3.10.7 Финален на експеримента и план за продължение

Проведеният експеримент изпълнява основната си цел: да валидира end-to-end TiDAR training/inference pipeline и да даде първа сравнителна картина за ефекта от различни loss конфигурации. В същото време резултатите показват, че при текущата постановка разделителната способност между вариантите е ограничена.

Основното наблюдение е силното припокриване на кривите между различните директории/конфигурации. В значителна част от диапазона разликите са минимални и на места практически неразличими визуално. Това е индикация, че в текущия режим доминира влиянието на корпуса, а не на конкретния избор на auxiliary loss.

Работната интерпретация е, че TinyStories е силно структуриран корпус с ограничена лексикална вариативност. При такава среда локална грешка в отделна позиция често не води до трайно разминаване по следващите токени, защото контекстът остава лесен за възстановяване. Поради това режими като `delta_masked` могат да изглеждат по-слаби в ранния диапазон (0 - 30k), без това задължително да означава по-нисък потенциал на самия метод в по-труден домейн.

Втори ключов фактор е мащабната разлика спрямо оригиналната експериментална среда на NVIDIA. Тук базовият тренировъчният корпус е приблизително 50 пъти по-малък, докато референтната постановка използва значително по-голям предварително обучен модел (Qwen 1.5B, също 50 пъти по-голям от нашия модел), който има много по-широка предтренировъчна база (30 000 пъти по-голяма). Това ограничение директно намалява чувствителността на експеримента към фини ефекти от loss комбинациите.

Планирано продължение до защитата (следващи 3 месеца):

- втори експериментален цикъл върху по-информативен корпус от типа OpenWebText/Wikipedia/куриран web text;
- увеличение на модела и тренировъчния обем, така че loss режимите да се разграничат по-ясно;
- избор на най-устойчивите конфигурации от този междинен етап и пренасяне към финален training диапазон от порядъка 300M-1B параметри и 5B-30B токена.

Следователно текущият експеримент се приема като междинен, но необходим етап: системата е валидирана, наблюдавани са ограниченията на малък/лесен корпус, и е дефиниран конкретен план за доразвитие преди финалната защита.

ЧЕТВЪРТА ГЛАВА

РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ

4.1 Инсталация

Базовият начин за стартиране е чрез Docker образите в CICD/Docker/.

Минимален сценарий:

```
docker run --pull always -d --gpus all \
  --name giant-training \
  --mount type=bind,source="$HOME/GIANT",target=/proj \
  bonanc/giant-training:latest
```

```
docker exec -it giant-training bash
```

За automatic-S3-sync workflow е наличен отделен образ CICD/Docker/giant-training-S3/, за който е нужна база като Minio или подобна в облака, за постоянно обновяване на данните. При слаб интернет от страна на developer машината това е не препоръчително. S3 tool-овете са достатъчни за базовия вариант.

4.2 Стартиране на средата и създаване на експеримент

Под “проект” в тази система се разбира комбинация от:

- глобална конфигурация (GIANT/v2/Global_Config.yml, TiDAR/Global_Config.yml);
- локална model/training конфигурация (Config.yml);
- директории за dataset/checkpoints/logs.

Първата стъпка е избор на data root и checkpoint root, след което се изпълняват pipeline и training скриптовете.

4.3 Подготовка и импорт на datasets

Импортиране на datasets.

Примери:

```
python GIANT/v2/data_pipeline/build_corpus.py \
  --config GIANT/v2/data_pipeline/Config.yml
python TiDAR/data_pipeline/Run_pipeline.py \
  --config TiDAR/data_pipeline/data_configs/Greedy_exp_500m.yml
```

Системата ще генерира Arrow shard-ове и manifest/statistics файлове.

4.4 Работа с training stages

Stage структурата играе ролята на curriculum по време на обучение.

4.4.1 Добавяне и пренареждане на stages

Еквивалент: добавяне и пренареждане на stages в YAML. Всеки stage може да задава seq_len, epochs, fraction и stage-local loss коефициенти.

4.4.2 Избор на checkpoint

Еквивалент: избор на конкретна checkpoint версия за инференс.

```
python GIANT/v2/model/Generate_faster.py \  
  --checkpoint latest  
python TiDAR/model/inference.py \  
  --checkpoint /proj/giant-data/TiDAR/checkpoints/params/step_0012100.npz
```

4.4.3 Разделяне на run на етапи

Еквивалент: разделяне на training run на етапи с различни objectives или подмяна на config по време на следващ stage.

4.4.4 Нулиране на експеримент

Еквивалент: reset на конкретен експеримент чрез нов checkpoint root или презаписване на stage output директория (pipeline регенерация).

4.5 Управление на изпълнението

Основни команди за runtime управление:

- старт на обучение: Run_training.py;
- пауза/спиране: SIGINT/SIGTERM (graceful stop след текущ chunk);
- resume: --resume latest или път до конкретна checkpoint;
- inference режими: greedy (--temperature 0) или sampling (--temperature > 0, --top_k).

4.6 Контрол на KV cache и inference мащаб

За инференс производителност Generate_faster.py поддържа KV cache bucket избор:

- автоматичен избор;
- конфигурационни buckets;
- CLI override чрез --kv_cache_buckets;
- изключване чрез --disable_kv_buckets.

Това позволява баланс между памет и latency, особено при различни prompt/steps комбинации.

4.7 Настройка на training параметри

Настройка на hyperparameters и loss коефициенти.

За TiDAR основните контроли са в `training.loss`:

- `alpha, beta`;
- `rho, chi`;
- `delta, delta_masked`;
- `eta, eta_T`;
- `gamma, gamma_topk`.

4.8 Бързи CLI команди

Работният процес е CLI-first. Типични бързи команди:

- `python GIANT/v2/model/Run_training.py --resume latest`
- `python GIANT/v2/model/Evaluate.py --checkpoint latest`
- `python GIANT/v2/model/Chat.py --chat --steps 128`
- `python GIANT/v2/model/Generate_faster.py --prompt "Hello" --steps 64 --verbose`
- `python TiDAR/model/Run_training.py --config TiDAR/model/Config_135m.yml`
- `python TiDAR/model/inference.py --prompt "Hello" --draft_len 8 --temperature 0.0`

Главният контейнер е базова runtime среда за обучение и инференс, която съдържа само необходимите системни и ML зависимости: CUDA runtime/драйверно-съвместими библиотеки, JAX/Flax/Optax/Orbx стек, Python tooling и build инструменти за възпроизводимо изпълнение на скриптовете. Така се гарантира, че training и inference поведението е консистентно между локална машина и remote GPU среда.

Репото и dataset/checkpoint данните не са вградени в самия Docker образ. Те се добавят след стартиране на контейнера чрез bind mount, sync или S3 workflow. Този подход държи образа по-малък, ускорява обновяването на кода и данните, и избягва ненужно преизграждане на image при всяка промяна в експериментите.

ЗАКЛЮЧЕНИЕ

В настоящата дипломна работа е разработена цялостна инженерна система за изграждане и експериментиране с decoder-only езикови модели, която обединява подготовка на данни, обучение, инференс и експериментална отчетност в единна codebase с CLI работен процес. Основният фокус е върху практическа възпроизводимост (конфигурации, checkpoints, resume), както и върху измеримо ускорение и сравнение на decode политики при реалистични ограничения на хардуера.

Постигнатите резултати могат да се обобщят в няколко ключови приноса:

- реализиран е стабилен data pipeline за ingest, стандартизиране, shard-ване и статистики върху корпуси, подходящ за повторяеми training цикли;
- реализирано е stage-базирано обучение на decoder-only Transformer с възможност за възстановяване на състояние и контролирани експерименти чрез конфигурации;
- реализиран е ускорен KV-cache инференс и benchmarking слой, позволяващ сравнение на throughput/latency при различни decode режими и параметри;
- изградена е инфраструктура за remote GPU изпълнение, Docker базирано стартиране и синхронизация на артефакти (datasets, checkpoints, отчети);
- реализирано е изследователско разширение (TiDAR/Anchor-TiDAR), което добавя специализирана masking логика и допълнителни loss термини за контрол на acceptance/quality поведението и за по-ефективно използване на паралелизма при decode.

Постигнатите резултати потвърждават, че архитектурното разделяне между стабилна базова платформа (GIANT) и експериментален слой (TiDAR) е обосновано: базовата част остава преизползваема за обучение и инференс, а изследователските промени могат да се развиват итеративно без да нарушават основния работен процес. Това улеснява едновременно инженерното поддържане и научния цикъл по формулиране на хипотеза, провеждане на експеримент и документиране на резултатите. Същата система е приложима и извън експериментален режим: моделът може да бъде обучен в рамките на платформата и използван директно за inference/serving без отделна миграция към друг стек.

Емпиричните резултати показват, че TiDAR постига ефективно съчетание на висока скорост и запазено качество чрез хибриден механизъм на ниво последователност: паралелно draft генериране и autoregressive верификация в рамките на един forward pass със структурирана attention маска. В проведените head-to-head измервания върху модели до 7B параметри е отчетено ускорение от 4.93x до 9.75x спрямо AR базовата линия, при стабилно inference поведение в различни GPU режими. Основното ограничение в текущата работа остава експериментът с различни loss термини, който е проведен в по-малък мащаб и следва да бъде разширен към по-големи модели и по-богати корпуси, за да се разграничат по-ясно фините ефекти между loss комбинациите.

Планирано бъдещо развитие:

- multi-GPU distributed обучение и по-големи тренировъчни диапазони;
- разширяване на корпуса към по-информативни домейни и по-строга evaluation методология (напр. Wikipedia, OpenWebText);
- интеграция на допълнителни архитектурни и serving оптимизации (напр. MoE¹⁹/MLA¹⁸, по-богати decode политики) и оценка на влиянието им върху Free Token Slots;
- разработване на мащабируем слой за обслужване на заявки (изпълнителен модул и управляващи скриптове), който приема паралелни заявки, групира ги динамично в общи партии и осигурява стабилно управление на KV cache;
- автоматизирано генериране на по-пълни отчети за quality/speed trade-off и стандартизирани сравнения между конфигурации.

Исползвана литература

- [1] Vaswani, A. et al. **Attention Is All You Need**. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- [13] Ba, J. L., Kiros, J. R., Hinton, G. E. **Layer Normalization**. 2016. <https://arxiv.org/abs/1607.06450>
- [10] Zhang, B., Sennrich, R. **Root Mean Square Layer Normalization (RMSNorm)**. 2019. <https://arxiv.org/abs/1910.07467>
- Srivastava, N. et al. **Dropout: A Simple Way to Prevent Neural Networks from Overfitting**. 2014. <https://jmlr.org/papers/v15/srivastava14a.html>
- [11] Su, J. et al. **RoFormer: Enhanced Transformer with Rotary Position Embedding**. 2021. <https://arxiv.org/abs/2104.09864>
- [12] Shazeer, N. **GLU Variants Improve Transformer**. 2020. <https://arxiv.org/abs/2002.05202>
- [6] Kingma, D. P., Ba, J. **Adam: A Method for Stochastic Optimization**. 2014. <https://arxiv.org/abs/1412.6980>
- [14] Loshchilov, I., Hutter, F. **Decoupled Weight Decay Regularization (AdamW)**. 2017. <https://arxiv.org/abs/1711.05101>
- [15] Dao, T. et al. **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. 2022. <https://arxiv.org/abs/2205.14135>
- Dao, T. **FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning**. 2023. <https://arxiv.org/abs/2307.08691>
- [16] Leviathan, Y., Kalman, M., Matias, Y. **Fast Inference from Transformers via Speculative Decoding**. 2023. <https://arxiv.org/abs/2211.17192>
- Stern, M. et al. **Blockwise Parallel Decoding for Deep Autoregressive Models**. 2018. <https://arxiv.org/abs/1811.03115>
- Cai, T. et al. **Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads**. 2024. <https://arxiv.org/abs/2401.10774>
- [17] Li, Y. et al. **EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty**. 2024. <https://arxiv.org/abs/2401.15077>
- NVIDIA Research. **Think in Diffusion, Talk in Autoregression (TiDAR)**. 2025. <https://arxiv.org/pdf/2511.08923>
- [8] Touvron, H. et al. **LLaMA: Open and Efficient Foundation Language Models**. 2023. <https://arxiv.org/abs/2302.13971>

- [5] Hoffmann, J. et al. **Training Compute-Optimal Large Language Models (Chinchilla)**. 2022. <https://arxiv.org/abs/2203.15556>
- Kaplan, et al. **Scaling Laws for Neural Language Models**. 2020. <https://arxiv.org/abs/2001.08361>
- [2] Kwon, W. et al. **Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM)**. 2023. <https://arxiv.org/abs/2309.06180>
- NVIDIA. **NVIDIA Ampere Architecture In-Depth**. 2020. <https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/>
- NVIDIA. **NVIDIA A100 Tensor Core GPU Architecture Whitepaper**. 2020. <https://resources.nvidia.com/en-us-tensor-core/nvidia-ampere-architecture-whitepaper>
- NVIDIA. **NVIDIA A40 Datasheet**. 2020. <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/documents/nvidia-rtx-a40-datasheet.pdf>
- JAX Documentation. <https://docs.jax.dev/>
- Flax Documentation. <https://flax.readthedocs.io/>
- Optax Documentation. <https://optax.readthedocs.io/>
- [4] Orbx Checkpointing Documentation. <https://orbx.readthedocs.io/>
- [3] Hugging Face Datasets Documentation. <https://huggingface.co/docs/datasets>
- Hugging Face Transformers Documentation. <https://huggingface.co/docs/transformers>
- Docker Documentation. <https://docs.docker.com/>
- Tailscale Documentation. <https://tailscale.com/kb>
- s5cmd Documentation. <https://github.com/peak/s5cmd>
- [9] Andrej Karpathy. **Let's build GPT: from scratch, in code, spelled out**. YouTube, 2023. <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- Andrej Karpathy. **The spelled-out intro to neural networks and backpropagation: building micrograd**. YouTube, 2022. <https://www.youtube.com/watch?v=VMj-3S1tku0>
- Typst Documentation. <https://typst.app/docs/>
- [7] Brutlag, J. **Speed Matters for Google Web Search**. Google, 2009. https://services.google.com/fh/files/blogs/google_delayexp.pdf
- [18] DeepSeek-AI. **DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model (MLA + MoE)**, 2024. <https://arxiv.org/abs/2405.04434>
- [19] DeepSeek-AI. **DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models**, 2024. <https://arxiv.org/abs/2401.06066>
- Beltagy, I., Peters, M. E., Cohan, A. **Longformer: The Long-Document Transformer** (sliding-window attention), 2020. <https://arxiv.org/abs/2004.05150>

СЪДЪРЖАНИЕ

УВОД	1
ПЪРВА ГЛАВА	2
ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ СИСТЕМИ, ПОДОБНИ НА GIANT	2
1.1 Базов контекст: какво е езиков модел	2
1.2 Термини (кратки дефиниции)	2
1.3 Сходни технологии и инженерни практики	3
1.3.1 LLaMA семейство (Meta)	3
1.3.2 SmolLM / компактни open модели	3
1.3.3 nanoGPT / minGPT	4
1.3.4 vLLM и TensorRT-LLM	4
1.3.5 Защо скоростта е критична за LLM приложения	4
ВТОРА ГЛАВА	5
ФУНКЦИОНАЛНИ ИЗИСКВАНИЯ И АРГУМЕНТАЦИЯ НА ИЗБОРА НА СРЕДСТВА	
ЗА РАЗРАБОТКА	5
2.1 Функционални изисквания	5
2.1.1 Входни източници и ingest pipeline	5
2.1.2 Подготовка на корпус, токенизация и stage-и	5
2.1.3 Конфигурационно управление на експерименти	6
2.1.4 Мониторинг и метрики по време на изпълнение	6
2.1.5 Управление на хиперпараметри и decode политика	6
2.1.6 Артефакти, checkpoint-и и отчетност	6
2.2 Аргументация за избор на езика и софтуерните средства	7
2.2.1 Избор на езика за програмиране	7
2.2.2 Избор на платформа за разработка	7
2.2.3 Избор на изчислителен фреймуърк	7
2.2.4 Избор на optimizer/checkpoint инструменти - Orbach, Optax и AdamW	8
2.2.5 Избор на средства за deployment и синхронизация	8
2.3 Структура на данните и вътрешен модел на системата	8
2.3.1 Формат на данните и логика на съхранение	8
2.3.2 Stage конфигурация и curriculum модел	9

2.3.3 Конфигурационни домейни на системата	9
2.3.4 Схема на dataset записите в shard	9
2.3.5 Източници и ingest адаптери	9
2.4 Цялостна архитектура и структурна организация	9
2.4.1 Интерфейсен слой (CLI)	10
2.4.2 Управляващи модули	10
2.4.3 Основни entry points	10
2.5 Основен математически модел на трансформер архитектурата	11
2.5.1 Теоретична отправна точка: Attention Is All You Need	11
2.5.2 Multi-head attention: защо са нужни “много глави”	13
2.5.3 От класически Transformer към decoder-only LLM	14
2.5.4 Нормализация, активации и позиционна информация в GIANT	16
2.5.5 Обучителна цел (AR) и връзка с TiDAR	18
2.6 Спекулативно декодиране (въведение преди TiDAR)	19
2.6.1 Защо single-user decode не използва добре GPU	20
ТРЕТА ГЛАВА	22
РЕАЛИЗАЦИЯ НА ПРИЛОЖЕНИЕТО	22
3.1 Реализация на потребителски интерфейс	22
3.1.1 Основни входни точки (CLI)	22
3.1.2 Наблюдение и визуализация на резултати	22
3.1.3 Управление на stage прогреса	22
3.1.4 Управление на параметри	22
3.1.5 Добавяне на входни корпуси	22
3.1.6 Експорт на артефакти	23
3.1.7 Примерен training run в терминал	23
3.2 Реализация на основните системни модули	25
3.2.1 ConfigLoaderManager	25
3.2.2 TrainLoopOrchestrator	25
3.2.3 DataLoaderManager	27
3.2.4 CheckpointManager	28
3.2.5 InferenceManager	28

3.2.6 RemoteOpsManager	32
3.3 Реализация на data pipeline	33
3.3.1 Документ като източник на токени	33
3.3.2 Преобразуване от документ към sequence прозорци	35
3.3.3 Директно четене и минимизиране на IO overhead	35
3.3.4 Шардване	35
3.3.5 Loader batch изграждане	35
3.3.6 Допълнителни preprocessing опции	35
3.4 Имплементация на “Think in Diffusion, Talk in AutoRegression”	36
3.4.0.1 Контекст: от класически speculative decode към TiDAR	36
3.4.0.2 Основна идея на TiDAR в един forward pass	37
3.4.0.3 Структурирани маски и достъп по позиции	37
3.4.0.4 Prefill маска	38
3.4.0.5 Training маска	39
3.4.0.6 Loss формулировка и баланс AR:Diff	40
3.4.0.7 Anchor-TiDAR (финален акцент)	40
3.4.1 Представяне на dual sequence вход	41
3.4.2 Преобразуване и маскиране по позиции	41
3.4.3 KV-cache pointer commit/rollback	42
3.4.4 Извличане на verify и predraft логити	42
3.4.5 Decode сценарий (Anchor-TiDAR)	43
3.4.6 Допълнителна визуална интерпретация за Mij	43
3.5 Експеримент: Free Token Slots	44
3.5.1 Native decode attention	44
3.5.2 Kernel terms (контролен анализ)	45
3.5.3 MLP scaling	45
3.5.4 Sampling path (greedy и non-greedy)	46
3.6 Резултати: TiDAR срещу AR (A40 и H100)	47
3.7 Разширена loss функция за TiDAR	47
3.7.1 Формулировка с независими коефициенти	47
3.8 Резултати от greedy run-ове (135M vs 360M)	49

3.9 Допълнителен експеримент: TinyStories attention head sweep	50
3.10 Проведен експеримент с различни loss функции и конфигурации	51
3.10.0 Избор на TinyStories и базова конфигурация	51
3.10.1 Експериментален протокол и избор на checkpoint	53
3.10.2 Сравнявани loss режими	54
3.10.3 Интерпретация на AR и Diff кривите	54
3.10.4 Greedy acceptance: основен таргет на sweep-a	55
3.10.5 Inference резултати (Accept/Iter таблица)	57
3.10.6 Локален механистичен анализ чрез logits хистограми	57
3.10.7 Финален на експеримента и план за продължение	60
ЧЕТВЪРТА ГЛАВА	61
РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ	61
4.1 Инсталация	61
4.2 Стартиране на средата и създаване на експеримент	61
4.3 Подготовка и импорт на datasets	61
4.4 Работа с training stages	62
4.4.1 Добавяне и пренареждане на stages	62
4.4.2 Избор на checkpoint	62
4.4.3 Разделяне на run на етапи	62
4.4.4 Нулиране на експеримент	62
4.5 Управление на изпълнението	62
4.6 Контрол на KV cache и inference мащаб	62
4.7 Настройка на training параметри	63
4.8 Бързи CLI команди	63
ЗАКЛЮЧЕНИЕ	64
Използвана литература	66